Course Title : HIGH PERFORMANCE COMPUTING Ass-8

Name : M.Abhishek

Batch no : 11

Hall Ticket No : 2303A51762

1. Lab: OpenMP Performance Analysis and Hotspot Detection

Objective:

To identify performance hotspots and understand basic profiling concepts**.**

Task 1: Compile with Timing

gcc -O2 -fopenmp sum_openmp.c -o sum_openmp

```
mithish@LAPTOP-A2FO3865:~/numaass$ cd ~
mithish@LAPTOP-A2FO3865:~$ ls
numaass  sum_openmp  sum_openmp.c  sum_serial  sum_serial.c
mithish@LAPTOP-A2FO3865:~$ gcc -O2 -fopenmp sum_openmp.c -o sum_openmp
mithish@LAPTOP-A2FO3865:~$ time ./sum_openmp
Parallel Sum: 249999997500000.000000
Execution Time: 0.057331 seconds
Threads Used: 8

real    0m0.067s
user    0m0.360s
sys     0m0.011s
mithish@LAPTOP-A2FO3865:~$
```

2. Task 2: Measure Execution Time

Use:

omp_get_wtime()

Record time for:

Serial execution

Parallel execution

```
GNU nano 7.2                                    sum_serial.c
#include <stdio.h>
#include <omp.h>

int main() {

    long long i, n = 1000000000;
    double sum = 0.0;

    double start = omp_get_wtime();   // Start time

    for(i = 0; i < n; i++) {
        sum += i;
    }

    double end = omp_get_wtime();     // End time

    printf("Serial Sum: %f\n", sum);
    printf("Serial Execution Time: %f seconds\n", end - start);

    return 0;
}
```

```
nithish@LAPTOP-A2FOJ865:/mnt/c/Users/Sreeshma$ nano sum_serial.c
nithish@LAPTOP-A2FOJ865:/mnt/c/Users/Sreeshma$ gcc -O2 -fopenmp sum_serial.c -o sum_serial
nithish@LAPTOP-A2FOJ865:/mnt/c/Users/Sreeshma$ ./sum_serial
Serial Sum: 499999999067108992.000000
Serial Execution Time: 5.416679 seconds
nithish@LAPTOP-A2FOJ865:/mnt/c/Users/Sreeshma$ |
```

```
  GNU nano 7.2                                   sum_serial.c
#include <stdio.h>
#include <omp.h>

int main() {
    long long i, n = 1000000000;
    double sum = 0.0;

    double start = omp_get_wtime();   // Start time

    #pragma omp parallel for reduction(+:sum)
    for(i = 0; i < n; i++) {
        sum += i;
    }

    double end = omp_get_wtime();     // End time

    printf("Parallel Sum: %f\n", sum);
    printf("Parallel Execution Time: %f seconds\n", end - start);
    printf("Threads Used: %d\n", omp_get_max_threads());

    return 0;
}
```

```
nithish@LAPTOP-A2FOJ865:~$ nano sum_serial.c
nithish@LAPTOP-A2FOJ865:~$ nano sum_serial.c
nithish@LAPTOP-A2FOJ865:~$ nano sum_serial.c
nithish@LAPTOP-A2FOJ865:~$ gcc -O2 -fopenmp sum_openmp.c -o sum_openmp
nithish@LAPTOP-A2FOJ865:~$ export OMP_NUM_THREADS=8
nithish@LAPTOP-A2FOJ865:~$ ./sum_openmp
Parallel Sum: 249999997500000.000000
Execution Time: 0.100298 seconds
Threads Used: 8
```

3. Task 3: Observation Questions

1. What is a hotspot in performance analysis?

When we talk about a *hotspot* in performance analysis, we mean the part of the program where most of the execution time is spent. It is the section of code that does the maximum work and consumes the most CPU time.

In our experiment (Task 1 and Task 2), we measured execution time for both serial and parallel versions of the summation program. After running the program and observing the timing results, it is clear that almost all the execution time is spent inside the **for loop** that performs the summation.

In Task 1, when we used the `time` command, the total execution time mainly depended on how long the loop took to execute.

In Task 2, when we used `omp_get_wtime()`, we again observed that the difference between start and end time mostly reflects the time taken by the loop.

Other parts of the program, like variable declarations and printing results, take very little time compared to the loop.

So, in our case, the **summation loop (serial and parallel loop)** is the hotspot because it runs billions of iterations and dominates the program's total execution time.

Identifying this hotspot is important because improving or parallelizing this loop (using OpenMP) directly improves the overall performance of the program.

2. Why does synchronization increase execution time?

In our OpenMP experiment, we compared serial and parallel execution times. The parallel version was faster because multiple threads worked at the same time. However, synchronization can reduce this benefit.

Synchronization increases execution time because it makes threads wait for each other. When several threads try to access or update the same shared variable (like `sum` in our program), the system has to control access to avoid errors. This control mechanism slows things down.

In our summation program, if we used a `critical` section instead of the `reduction` clause, every thread would have to wait before updating the `sum` variable. Even though multiple threads are running, only one would be allowed to update the value at a time. This waiting reduces parallel efficiency and increases execution time.

So, even though parallel programming aims to speed up execution (as seen in Task 1 and Task 2), synchronization adds overhead. The more threads wait for shared resources, the less performance improvement we get.

In simple terms, synchronization increases execution time because it reduces the freedom of threads to run simultaneously.

3. Why does speedup stop increasing after some threads?

When we increased the number of threads in our OpenMP program, we noticed that execution time reduced at first. This gave us good speedup. But after some point, adding more threads did not improve performance much. Sometimes, it may even slightly reduce performance.

This happens because not all parts of the program can be parallelized. In our summation program, the main loop is parallel, but other parts like variable setup, thread management, and final result combination still take time. These small serial portions limit the maximum possible speedup.

Another reason is overhead. When we increase threads, the system has to create, manage, and synchronize them. This coordination takes extra time. After a certain number of threads, the overhead becomes comparable to the actual computation time.

Also, the number of CPU cores in the system matters. If your system has 8 cores and you use more than 8 threads, they must share cores. This leads to context switching and resource contention, which reduces performance gains.

So, in our Task 1 and Task 2 experiments, speedup improved when we increased threads up to a certain point. After that, it stopped increasing significantly because of:

- Serial portions of the program
- Thread management overhead
- Synchronization costs
- Hardware limitations (limited CPU cores)

In simple terms, more threads do not always mean more speed. After a limit, the system cannot efficiently use additional threads